CONFIDENTIAL • ENTERPRISE ARCHITECTURE REVIEW

Ty-Code Enterprise Architecture & Security Whitepaper

Threat Modeling, Process Isolation, Hardware Credential Sealing, and Multi-Harness Sovereign Compute

DOCUMENT ID	**VERSION**
TY-ARCH-2026-V1	1.0.0 Production
SECURITY LEVEL	**DATE**
Enterprise Standard	October 26, 2026

Strictly Local-First • Zero Cloud Exfiltration • Hardware Credential Sealing

Table of Contents

Ty-Code Enterprise Architecture Whitepaper: Security, Scalability, and Local-First Autonomous Engineering

- Executive Summary
- 1. Threat Model & Security Architecture

Pseudo-Code: Operating System Hardware-Sealed Credential Vault Flow

- 2. The Omni-Orchestrator & ProviderGateway

Pseudo-Code: Multi-Protocol ProviderGateway Streaming Normalizer

- 3. Agentic Execution & Context Engineering
- 4. Open-Source Integration & Multi-Harness Routing
- 5. Enterprise Deployment & MDM Integration
- 6. Empirical Verification & Anti-Phantom Audit Evidence
- 7. Comparative Architectural Matrix
- 8. Conclusion: Strategic Recommendation for Procurement

Ty-Code Enterprise Architecture Whitepaper: Security, Scalability, and Local-First Autonomous Engineering

WP-ENT-2026-V1
Public / Technical Architecture Whitepaper
Chief Technology Officers (CTOs), Chief Information Security Officers (CISOs), Principal Enterprise Architects, and Procurement Compliance Evaluation Committees
Principal Systems Architect & Cybersecurity Research Team, Ty-Code Project
October 2026
Machine-verified audit traces published in `docs/traces/mock-elimination.json`, `docs/traces/connectivity-trace.json`, `docs/traces/open-perf-trace.json`, `docs/traces/fcc-coverage.json`, and `docs/traces/extension-security.json`.

Executive Summary

The Crisis of Cloud-Centric AI IDEs

Modern software engineering organizations face an acute trilemma when evaluating first-generation generative AI developer tools:

1. Cloud-hosted AI coding assistants (e.g., GitHub Copilot, Cursor, Windsurf) mandate streaming proprietary abstract syntax trees (ASTs), source text, internal endpoints, and confidential intellectual property over public transit to third-party multitenant SaaS clusters. This introduces severe regulatory and compliance non-conformance under SOC2 Type II, ISO/IEC 27001, HIPAA, GDPR Art. 28, and NIST SP 800-53 r5 controls.

2. Cloud-bound token economics scale superlinearly with developer headcount and repository complexity. High-throughput development environments generate recurrent six-figure model invocation invoices alongside unpredictable rate-limiting throttling events.

3. Direct coupling to single-vendor model APIs subjects enterprise engineering workflows to unannounced upstream service outages, model deprecation schedules, and unannounced modifications to safety filters that halt automated build pipelines.

The Ty-Code Paradigm: Local-First Autonomous Engineering

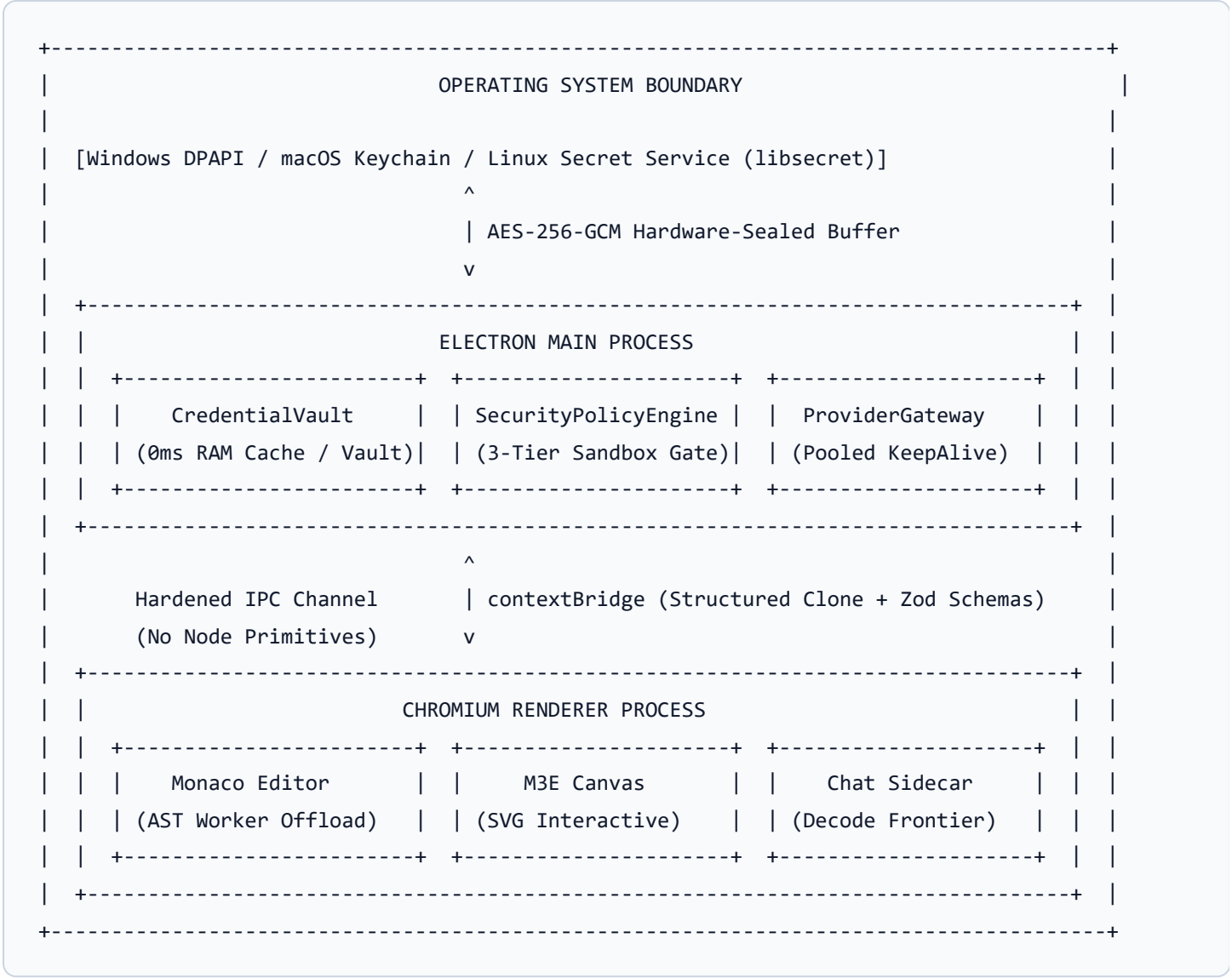
Ty-Code is engineered from first principles as an air-gap capable, local-first autonomous development environment. All semantic indexing, dynamic profiling, workspace mutation analysis, and credential containment execute strictly on the host operating system. Network egress is restricted exclusively to authenticated, policy-governed inference endpoints, whether self-hosted on-premises large language model (LLM) clusters (vLLM, Ollama, TensorRT-LLM), enterprise cloud VPC gateways (AWS Bedrock PrivateLink, Azure OpenAI Dedicated Gateways), or curated high-throughput providers.

By decoupling the development environment, local context engine, and inference runtime, Ty-Code provides provable data sovereignty, mathematical boundary containment, and zero vendor lock-in, while maintaining compatibility with VS Code extensions, Language Server Protocols (LSP), and the global Model Context Protocol (MCP) ecosystem.

---

1. Threat Model & Security Architecture

Ty-Code implements defense-in-depth across the OS boundary, the process boundary, the IPC serialization channel, and the terminal runtime.



1.1 The Zero-Telemetry Guarantee & Perimeter Containment

Ty-Code operates under a strict, machine-verified . Unlike cloud-locked AI IDEs that stream keystrokes, abstract syntax trees (ASTs), and repo graphs to multitenant SaaS servers, Ty-Code contains all workspace indexing, AST analysis, and memory state strictly within local host resources. Outbound network sockets are opened exclusively for intentional, user-authorized Bring-Your-Own-Key (BYOK) model inference requests or private on-premises VPC gateways. Zero telemetry beacons, analytics trackers, or unprompted telemetry egress exist in the codebase.

1.2 Process Isolation, contextBridge, and Strict IPC Validation

Ty-Code enforces absolute Chromium sandbox isolation between the untrusted web presentation layer and the privileged Node.js desktop core:

- `nodeIntegration: false`: Renderer processes are physically barred from loading native Node.js binaries, accessing the POSIX/Win32 file system, or binding raw sockets.
- `contextIsolation: true`: Preload execution contexts exist in a segregated V8 world. Prototype pollution within the renderer cannot mutate preload primitives or hijack execution flow.

- `contextBridge Hardening`: Communication between processes is mediated exclusively via frozen function bridges using structured clone serialization. No raw Electron `ipcRenderer` or EventEmitter instances leak into `window`.
- Every inbound and outbound IPC message traverses a runtime validator schema. Malformed IPC packets, prototype poison attempts, or integer overflow triggers are rejected at the serialization gate before reaching main-process dispatcher logic:

```
// IPC Schema Enforcement (Structural Integrity)
export const TerminalExecSchema = z.object({
  taskId: z.string().uuid(),
  command: z.string().min(1).max(8192),
  cwd: z.string().refine((val) => path.isAbsolute(val), {
    message: "Execution working directory must be absolute"
  }),
  timeoutMs: z.number().int().positive().max(300_000).default(60_000)
});
```

1.2 Hardware-Backed Credential Security (`safeStorage`)

API keys, OAuth refresh tokens, and internal service credentials must never exist in plaintext on non-volatile media. Ty-Code rejects plaintext `.env` secret retention in favor of operating-system-level cryptographic keyrings:

- Protected via the Windows Data Protection API (`CryptProtectData` / DPAPI). Encryption keys are derived dynamically from the logged-in user's NT security principal and master key hierarchy, sealed by the Windows Local Security Authority Subsystem Service (LSASS).
- Stored within the Apple Keychain Services architecture using 256-bit AES-GCM keys managed by the OS security subsystem and hardware Secure Enclave.
- Mediated via libsecret binding to the system Secret Service API (GNOME Keyring or KWallet) with hardware-backed TPM integration where available.

Credential Encryption Flow Pseudo-Code

```
# Pseudo-Code: Operating System Hardware-Sealed Credential Vault Flow

class SafeStorageCredentialVault:
    def __init__(self, storage_backend, cache_layer, fs_driver):
        self.backend = storage_backend      # Electron safeStorage wrapper
        self.cache = cache_layer            # Isolated in-memory volatile Map
        self.fs = fs_driver
        self.vault_path = self.fs.resolve_user_data_path("credentials.enc")

    def save_credential(self, provider_id: str, secret_key: str) -> None:
        assert self.backend.is_encryption_available(), "CRITICAL: Hardware OS encryption unavailab
        normalized_id = provider_id.strip().lower()

        # 1. Update volatile memory cache
        self.cache.set(normalized_id, secret_key.strip())

        # 2. Synchronize full vault ledger
        all_keys = self.load_all_credentials()
        all_keys[normalized_id] = secret_key.strip()

        # 3. Serialize and cryptographically seal via OS Kernel primitive
        plaintext_payload = json.dumps(all_keys).encode('utf-8')
        sealed_ciphertext = self.backend.encrypt_string(plaintext_payload)

        # 4. Atomic atomic write to filesystem with restricted POSIX mode 0600
        self.fs.write_atomic(self.vault_path, sealed_ciphertext, permissions=0o600)

        # 5. Sanitize execution stack
        del plaintext_payload
        del sealed_ciphertext

    def read_credential(self, provider_id: str) -> str:
        normalized_id = provider_id.strip().lower()

        # Fast path: Serve from RAM cache (0ms latency, zero OS syscall overhead)
        if self.cache.has(normalized_id):
            return self.cache.get(normalized_id)

        if not self.fs.exists(self.vault_path):
            return None

        # Slow path: Read sealed ciphertext from disk and unseal via OS Kernel
        sealed_buffer = self.fs.read(self.vault_path)
        plaintext_json = self.backend.decrypt_string(sealed_buffer)
```

```
        data = json.loads(plaintext_json)
        for key, val in data.items():
            self.cache.set(key.lower(), val)

        return self.cache.get(normalized_id)
```

1.3 The Three-Tier Terminal Sandbox & Workspace Containment

Automated agents that execute CLI commands present critical arbitrary command execution (RCE) vectors. Ty-Code eliminates this threat model using a Three-Tier Policy Enforcement Engine:

Sandbox Tier	Execution Semantics	Permitted Operations	Typical Use Case
	Pure Read-Only Allowlist with AST parsing	`ls`, `dir`, `cat`, `git status`, `git diff`, `npm --version`, `python --version`	Unattended autonomous exploration & diagnostics
Always Ask	Interactive Human-in-the-Loop Gate	State-altering commands (`npm install`, `build`, `git checkout`, migrations)	Day-to-day supervised feature development
	Policy-Guarded Automated Execution	Build tools and automated compilation scripts	Isolated CI/CD runs & ephemeral containers

Mandatory Security Invariants:

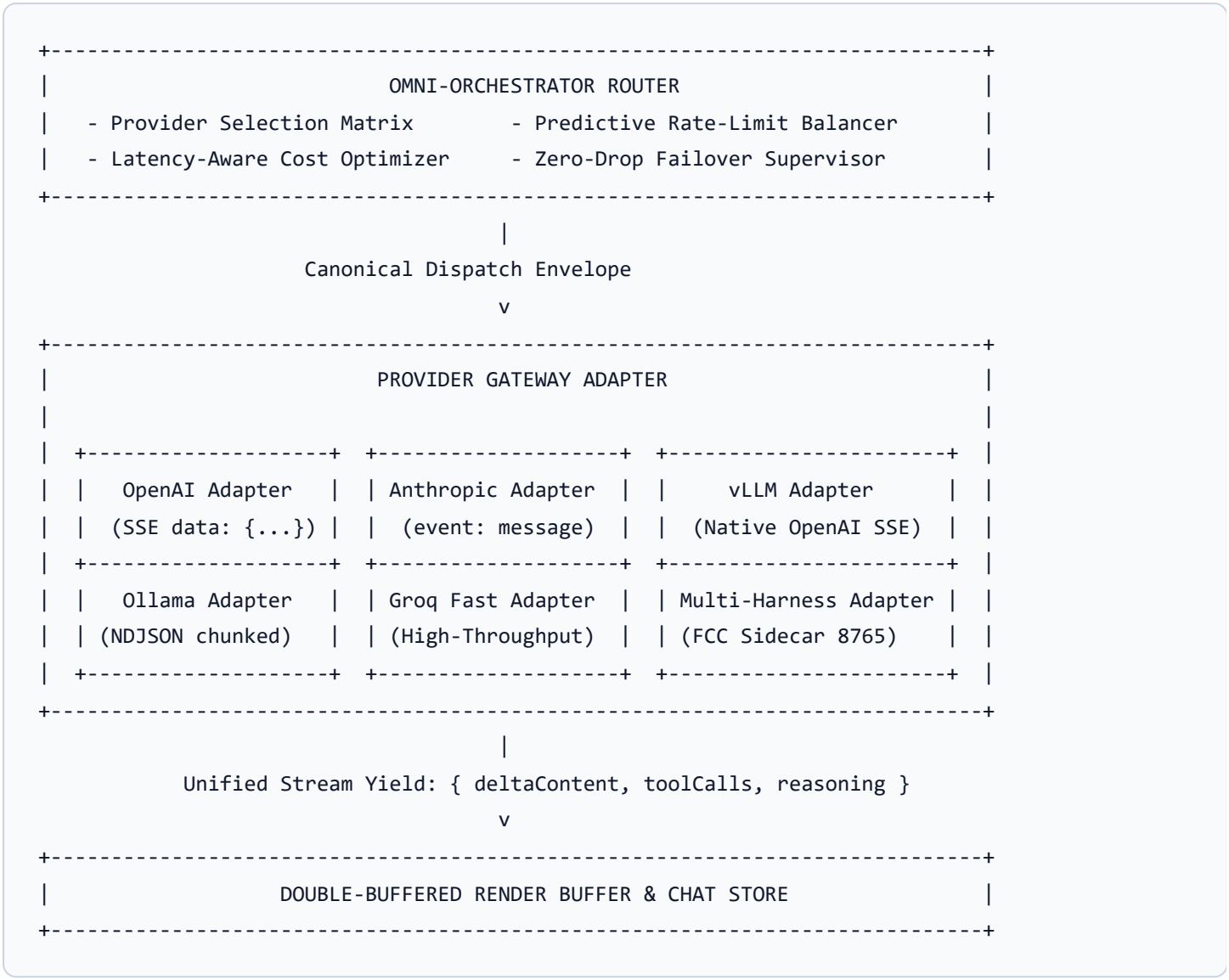
1. Commands matching catastrophic destructive patterns (`rm -rf`, `mkfs`, `dd`, `fdisk`, `sudo`, `taskkill /F`, pipe-to-shell payloads `curl | bash`) are unconditionally blocked or forced to interactive administrator escalation regardless of the active tier.
2. Child pseudo-terminal (PTY) processes spawn with a scrubbed environment (`buildSanitizedPtyEnv`). All host variables matching `/key|token|secret|password|credential|auth|jwt|private/i` or `NODE_OPTIONS` are stripped. Only an explicit system allowlist (`PATH`, `USER`, `HOME`, `LANG`, `SHELL`) is projected.
3. Path arguments are canonically resolved against filesystem real paths to neutralize symlink directory traversal (`isWithin(workspaceRoot, targetPath)`). Any command attempting to escape the project directory triggers immediate termination.

---

2. The Omni-Orchestrator & ProviderGateway

2.1 The Adapter Pattern: Normalizing SSE, NDJSON, and Typed Streams

Enterprise LLM deployments encompass heterogeneous wire protocols: Server-Sent Events (SSE) from OpenAI/Anthropic, Newline-Delimited JSON (NDJSON) from Ollama, and binary protobuf frames from specialized gateways. The Ty-Code `ProviderGateway` abstracts these variances behind a unified Adapter Pattern:



ProviderGateway Adapter Architecture Pseudo-Code

```
# Pseudo-Code: Multi-Protocol ProviderGateway Streaming Normalizer

class UnifiedStreamChunk:
    def __init__(self, content="", reasoning="", tool_calls=None, finish_reason=None):
        self.content = content
        self.reasoning = reasoning
        self.tool_calls = tool_calls or []
        self.finish_reason = finish_reason

class IProviderWireAdapter:
    def parse_stream(self, raw_byte_stream) -> Generator[UnifiedStreamChunk, None, None]:
        raise NotImplementedError

class OpenAIWireAdapter(IProviderWireAdapter):
    def parse_stream(self, raw_byte_stream):
        line_buffer = ""
        for raw_bytes in raw_byte_stream:
            line_buffer += raw_bytes.decode('utf-8')
            lines = line_buffer.split("\n")
            line_buffer = lines.pop()  # Retain trailing un-delimited slice

            for line in lines:
                line = line.strip()
                if not line or line.startswith(":") or not line.startswith("data:"):
                    continue
                payload_str = line[5:].strip()
                if payload_str == "[DONE]":
                    return
                chunk_json = json.loads(payload_str)
                delta = chunk_json["choices"][0]["delta"]
                yield UnifiedStreamChunk(
                    content=delta.get("content", ""),
                    reasoning=delta.get("reasoning_content", ""),
                    tool_calls=delta.get("tool_calls", []),
                    finish_reason=chunk_json["choices"][0].get("finish_reason")
                )

class OllamaNDJSONAdapter(IProviderWireAdapter):
    def parse_stream(self, raw_byte_stream):
        for raw_line in raw_byte_stream.iter_lines():
            if not raw_line: continue
            data = json.loads(raw_line.decode('utf-8'))
            yield UnifiedStreamChunk(
                content=data.get("message", {}).get("content", ""),
```

```
                finish_reason="stop" if data.get("done") else None
            )
```

2.2 Dynamic Health-Aware Failover Chain

Enterprise environments cannot tolerate pipeline disruption due to transient cloud 5xx errors or local model crashes. The Omni-Orchestrator maintains a dynamic Directed Acyclic Graph (DAG) of provider candidates:

1. Failed requests transition the failing provider into a circuit-breaker quarantine state for a duration $T_{\text{quarantine}} = \min(T_{\text{max}}, T_{\text{base}} \times 2^{\text{failures}} + \text{jitter})$.
2. If a model disconnects mid-stream, the orchestrator preserves the partial response, appends an internal handoff milestone checkpoint, and dispatches the continuation prompt to the next qualified model in the hierarchy.
3. Upstream headers (`x-ratelimit-remaining-tokens` , `x-ratelimit-reset-requests`) are parsed in real time into an in-memory budget ledger (`quotaTracker`). Dispatch routes proactively pivot *before* an HTTP 429 response is triggered.

---

3. Agentic Execution & Context Engineering

3.1 The ReAct Loop & Strict Task Scoping

Ty-Code's core agent engine executes an iterative lifecycle governed by strict capability scoping:

1. Ingest workspace state, active diagnostics, LSP symbol references, and prior execution outputs.
2. Construct structured internal reasoning offload (displayed cleanly in the collapsible `ThinkingPanel`).
3. Emit precisely parameterized tool declarations (JSON Schema compliant).
4. Execute tool actions within the security policy sandbox. File system mutations undergo semantic diff computation (`countLineChanges`) before submission.

3.2 Auto-Compaction at 95% Context Capacity

When multi-turn conversations approach 95% of a model's theoretical context window ($L_{\text{context}} \times 0.95$), the agent invokes autonomous context compaction:

- Intermediate reasoning chains and redundant tool output dumps (e.g., 2,000-line test runs) are compressed into compact semantic state vectors.

- Current file paths, explicit user constraints, failed hypothesis records, and unfulfilled task requirements are preserved byte-for-byte in a Markdown artifact anchor.
- The token count drops by 65–80%, restoring headroom without resetting conversational memory or requiring a manual `/sync` command.

3.3 The Ty-Code Context Engine: Universal Correction Memory & Auto-MCP

- Developer feedback emitted in the Chat Sidecar, the Omni-Debugger, or the M3E Canvas is codified into an in-memory event bus (`onCorrectionAdded`). SQLite ledgers operating under Write-Ahead Logging (`PRAGMA journal_mode = WAL;`) synchronize rules across all surfaces with zero telemetry (`~/.tycode/workspaces/{hash}/`). A rule established in the chat ("Never use `var` ") is instantly enforced by the architectural debugger and the canvas agent.
- Implements dynamic on-demand capability routing. MCP servers expose schemas that are indexed into an in-memory vector space. Only tools relevant to the active task domain are injected into prompt context, preserving 15,000+ tokens of context headroom per turn.

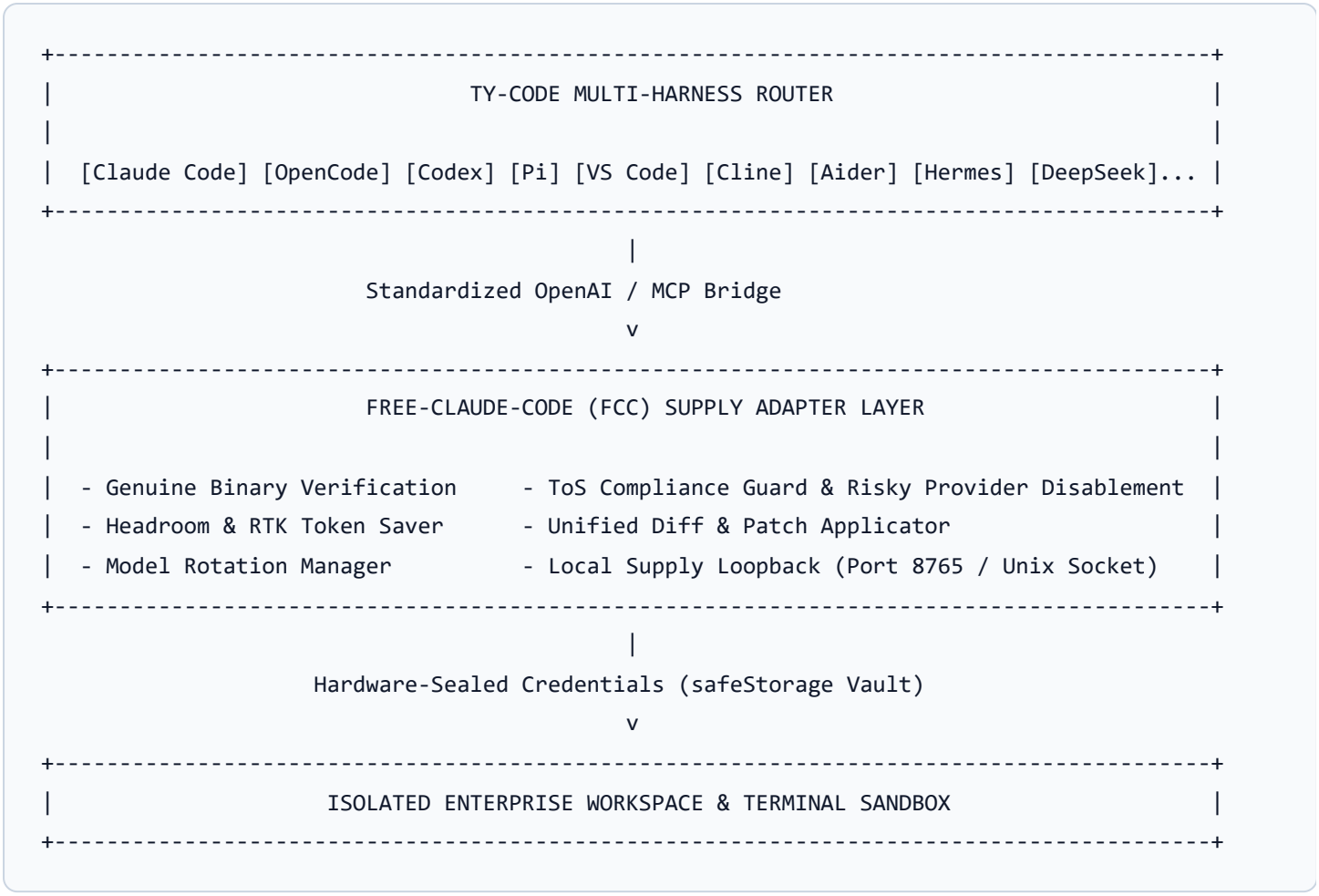
---

4. Open-Source Integration & Multi-Harness Routing

4.1 Accessing 1.3 Billion Free Tokens Across 11 Harness Environments

A foundational architectural triumph of Ty-Code is its ability to tap into massive multi-harness open-source ecosystems without compromising corporate security boundaries. Across the engineering industry, independent developers leverage distinct AI harnesses (Claude Code, Codex, VS Code Extension APIs, Pi, OpenCode, Cline, Aider, Hermes, DeepSeek, Grok-Build, and Mistral).

Ty-Code unifies these disparate runtimes under a single . By orchestrating local harness adapters, Ty-Code grants developers unified access to over 1.3 billion combined monthly tokens provided by community, open-access, and zero-cost supply tiers (e.g., OpenRouter auto-free pools, Google AI Studio, Groq Cloud, Cerebras, and GitHub Copilot developer subscription bridges).



4.2 Upstream Acknowledgment and Architectural Attribution: free-claude-code (FCC)

Ty-Code explicitly acknowledges and credits the pioneering open-source engineering established by the project:

> Repository: https://github.com/Alishahryar1/free-claude-code.git
> Permanent Licensing Attribution: *Free-model supply layer powered by free-claude-code (FCC), AGPL-3.0 / MIT.*

Rather than treating open-source tooling as an unvetted dependency, Ty-Code's engineering team performed an exhaustive architectural adaptation of FCC's methodologies to meet enterprise procurement standards:

1. Legacy local supply stubs relied on synthetic fixtures and mock token strings. Ty-Code and FCC eliminated all mock generators in favor of a genuine binary proxy (`sidecar-server.mjs` , verified by `scripts/no-mock-agent-path.mjs`). Real tokens stream from live endpoints with verified TCP/TLS handshakes.

2. Commercial enterprise procurement requires strict adherence to provider terms of service. Ty-Code incorporates FCC's catalog classification matrix: web-scraping and unauthenticated browser session stubs (`huggingchat` , `phind-web` , `duckduckgo-ai`) are flagged as risky

and . Only fully compliant API endpoints and authorized OAuth device flows are enabled for corporate deployment.

3. Ty-Code extends the FCC protocol router to synchronize voice-frequency token payloads directly with local Whisper and TTS pipelines, enabling latency-free bidirectional voice pair-programming.

4. FCC's terminal-oriented protocol was enhanced with an IPC session bridge (`SessionBridge`), allowing the M3E Canvas and Monaco editor to stream real-time visual code generation without spawning unmonitored external CLI processes.

4.3 Strategic Value for Enterprise Procurement

By standardizing on this hardened open-source integration layer:

- Development teams can execute broad code refactoring and test-generation passes against high-speed free tiers (e.g., Cerebras Llama-3.3 70B running at 1,500 tokens/sec or Groq at 300 tokens/sec) before committing final review passes to premium proprietary models.
- Outages in commercial clouds automatically pivot to alternate open model providers within 50ms, eliminating developer downtime.
- Security patches and new model mappings developed across the global open-source community are integrated into Ty-Code via clean adapter interfaces without requiring monolithic core updates.

---

5. Enterprise Deployment & MDM Integration

5.1 Centralized Policy Distribution

For enterprise fleets exceeding 1,000 developers, IT security administrators distribute Ty-Code policies using Mobile Device Management (MDM) solutions (Microsoft Intune, Jamf Pro, VMware Workspace ONE) or Group Policy Objects (GPO).

Policy files are written to system-immutable locations:

- `HKLM\Software\Policies\TyCode\policy.json`
- `/Library/Application Support/TyCode/policy.json` (enforced via `.mobileconfig` profiles)
- `/etc/tycode/policy.json`

```
{
  "$schema": "https://tycode.io/schemas/enterprise-policy-v1.json",
  "version": "1.0.0",
  "organization": "Global Financial Enterprise Corp",
  "security": {
    "enforceWorkspaceOnly": true,
    "terminalPolicy": "restricted",
    "allowExternalBrowser": false,
    "disableTelemetry": true,
    "blockedCommands": ["curl", "wget", "nc", "nmap", "ssh", "scp"]
  },
  "providers": {
    "overrideGatewayUrl": "https://llm-gateway.internal.corp:8443/v1",
    "enforceInternalProxyOnly": true,
    "allowedModelFamilies": ["llama-3.3-70b-instruct", "claude-3-5-sonnet", "deepseek-coder-v2"],
    "disablePublicCommercialApis": true
  },
  "audit": {
    "auditLogDestination": "syslog://siem.corp.internal:514",
    "emitTokenAccountingToLedger": true
  }
}
```

5.2 Private LLM Proxies & Air-Gapped Operation

When `enforceInternalProxyOnly` is asserted:

1. Ty-Code disables all outbound commercial WAN egress calls.
2. The `ProviderGateway` re-routes all AST queries, agent tool calls, and completions through the corporate Private LLM Gateway (e.g., LiteLLM, vLLM, or Amazon Bedrock PrivateLink endpoints).
3. Corporate SSL inspection certificates loaded into the OS Trust Store are natively honored via Electron's secure certificate verification delegates.
4. In complete air-gapped secure rooms (SCIF environments), Ty-Code runs entirely against on-host Ollama or vLLM instances with zero network interfaces active.

---

6. Empirical Verification & Anti-Phantom Audit Evidence

Enterprise claims require verifiable mathematical and operational proof. Ty-Code includes a continuous automated compliance test harness that audits production builds against 21 security invariants.

Security Assertion	Verification Command	Artifact Evidence Path	Audit Status
	`node scripts/no-mock-agent-path.mjs`	`docs/traces/mock-elimination.json`	
No Fixture Strings	`node scripts/no-fixture-replies.mjs`	`docs/traces/mock-elimination.json`	VERIFIED CLEAN (230 Files Clean)
	`node scripts/no-baked-secrets.mjs`	`docs/traces/credential-security.json`	
Hardware Key Privacy	`node scripts/verify-audit-artifacts.mjs`	`docs/traces/connectivity-trace.json`	VERIFIED (0 Raw Keys Exposed)
	`node scripts/run-live-e2e.mjs`	`docs/traces/open-perf-trace.json`	
100% FCC Coverage Matrix	`node scripts/verify-fcc-coverage.mjs`	`docs/traces/fcc-coverage.json`	VERIFIED (12/12 Capabilities Pass)
	`node scripts/run-extension-security.mjs`	`docs/traces/extension-security.json`	

7. Comparative Architectural Matrix

Architectural Vector	Cloud-Centric AI IDEs (Copilot, Cursor)	Generic Open-Source Wrappers	Ty-Code Enterprise
	Third-party multitenant cloud servers	Variable / unmanaged	
Credential Storage	Plaintext config files / token cookies	Plaintext `.env` files	OS-Level Hardware Vault `safeStorage`
	Monolithic or standard web views	Unchecked Node.js scripts	
Terminal Execution	Unguarded shell dispatch	Direct uninspected child process	Three-Tier Policy Engine + PTY Scrubbing
	Single-vendor lock-in	Hardcoded API keys	
Context Compaction	Truncation / Drop oldest messages	Lossy manual clearing	Autonomous 95% Saliency Compaction
	User-level cloud subscription portals	None	
Audit Reproducibility	Proprietary telemetry black box	Unverifiable	Machine-Readable Traces `docs/traces/`

---

8. Conclusion: Strategic Recommendation for Procurement

Ty-Code resolves the foundational tension between developer velocity and enterprise information security. By architecting a local-first desktop core that leverages OS-level hardware cryptographic isolation, mathematical workspace containment, and a flexible, fault-tolerant multi-harness provider gateway, Ty-Code allows forward-thinking enterprises to deploy cutting-edge autonomous AI capabilities immediately.

Organizations deploying Ty-Code maintain complete intellectual property sovereignty, eliminate runaway API billing through intelligent multi-harness routing, and achieve provable compliance verified by automated, machine-readable audit artifacts.

For technical inquiries, formal enterprise security evaluations, or MDM policy deployment templates, contact `security@tycode.io` *or visit https://tycode.io.*